

Prova de Conceito

Projeto Integrador 2025 (Des)conecta

Gustavo Eufrausino de Medeiros - 5062

Sumário

1- Visão Geral.....	3
2 - Decisões Técnicas.....	3
3 - Versões das tecnologias:.....	4
4 - Implementação:.....	5
Back-end.....	5
Front-end:.....	6
Amarrando tudo junto:.....	7
Implantação:.....	8
Conclusão:.....	9

1- Visão Geral

O presente documento aborda a arquitetura desenvolvida para a prova de conceito da aplicação (Des)conecta. Foram decididas quais tecnologias, frameworks e bibliotecas principais serão utilizadas durante o desenvolvimento do sistema. Em adição, é detalhado todo o processo de tomada de decisão de cada tecnologia.

O (Des)conecta consiste em uma aplicação web para ensinar o como informações são obtidas a partir de dados a partir de um modelo de storytelling e resolução de puzzles, por conta disso, foram selecionadas tecnologias que facilitassem o desenvolvimento de telas cativantes e que garantissem uma boa experiência de uso ao usuário. Tendo em vista o comportamento dos usuários pela aplicação (cliques), foi desenvolvida uma aplicação que contabilizasse a quantidade de cliques (guardando-as em um banco de dados remoto) e resgatasse essa quantidade também do banco, caso a quantidade de cliques seja maior ou igual a 10, o usuário pode clicar em um botão para trocar de tela. Apesar da proposta humilde, ela contempla as principais necessidades do software.

As tecnologias usadas para o desenvolvimento dessa aplicação foram: docker (infra e padronização), react + vite + tailwind css (front-end), spring boot + maven (back-end), MySQL (banco de dados remoto). As escolhas dessas tecnologias serão justificadas mais adiante neste documento.

O diagrama de implantação referente a esta prova de conceito está no documento Diagrama de Implantação presente na mesma pasta que este documento no repositório. Essa separação foi realizada com o intuito de preservar a qualidade da imagem e realizar comentários mais aprofundados sobre o diagrama lá, enquanto este se aprofunda nas tecnologias e decisões feitas para a construção da aplicação.

2 - Decisões Técnicas

Primeiramente, foi selecionado um modelo arquitetural para a aplicação, após estudar sobre os modelos: Layer, MVC, microsserviços, entre outros. Acreditei que o modelo MVC (errata - o modelo usado na verdade é o BCE) se adequasse melhor às necessidades da aplicação e ao tempo de treinamento da equipe de desenvolvimento. Após escolher o modelo, foi preciso definir as tecnologias necessárias para implementação da aplicação tendo em vista as restrições definidas pelo próprio projeto integrador: java para back-end, MySQL para banco de dados - sendo este banco de dados remoto.

Iniciando pela parte de infraestrutura, o [Docker](#) foi selecionado em virtude de possuir uma comunidade fortemente ativa e apresentar diversos materiais de estudo, além de apresentar uma curva de aprendizado não muito grande e facilitar a padronização dos ambientes de desenvolvimento.

Sequencialmente, como citado anteriormente, java é mandatório para o back-end, mas para acelerar o desenvolvimento, o framework [spring boot](#) foi escolhido por possuir uma comunidade ativa e muitos materiais de estudo, mas também por ser “orientado” para o desenvolvimento de APIs RESTful, mas também o ORM implementado através do JPA, o que torna menos custoso para os desenvolvedores trabalharem com banco de dados. Ademais o

[maven](#) tornou muito mais simples a adição de dependências aos back-end, assim como a realização de testes automatizados, compilação e execução da aplicação. Aliando o maven e o spring, o desenvolvimento foi muito menos trabalhoso do que seria, caso fosse necessário “inventar a roda novamente”. Para criação da estrutura do back-end, foi utilizado o site [spring inicializr](#), neste site, é possível gerar o pom.xml com todas as dependências necessárias ao back-end. Estas dependências foram: **Spring Boot DevTools** (hot reload entre outras funcionalidades para desenvolvimento web), **Spring Web** (tomcat entre outras funcionalidade úteis para desenvolvimento de APIs REST que usam MVC), **MySQL Driver** (ORM com MySQL), **Spring Data JPA** (ORM) e **Lombok** (fornece notações que simplificam o desenvolvimento, por exemplo, criação de construtores de forma automática).

Já o desenvolvimento do front-end foi elaborado tendo como base o [vite](#), o que tornou consideravelmente mais simples a criação da estrutura do front-end. O front-end é implementado usando [react](#), uma biblioteca javascript muito utilizada no mercado de desenvolvimento de aplicações web, assim como as tecnologias anteriores, possui uma curva de aprendizado suave e muitos materiais de estudo. Além do react, também foi usado o framework CSS [tailwind](#) para estilização das telas. O tailwind é frequentemente usado em projetos que incluem o react, para desenvolvedores front-end com baixa experiência, o tailwind facilita muito a estilização de páginas, além disso há muitos materiais de estudo sobre essa tecnologia.

O banco de dados foi disponibilizado por servidores da própria universidade, sendo preciso apenas conectar a aplicação nas portas corretas.

3 - Versões das tecnologias:

As versões das tecnologias usadas para esta prova de conceito são especificadas nos arquivos: pom.xml na pasta backend, Dockerfile tanto na pasta backend quanto na front-end e no arquivo package.json na pasta front-end. As versões das tecnologias estão listadas abaixo:

- Dockerfile (back-end):
 - Maven 3.9.9
 - Java 17 (era 21, mas por conta do deploy foi preciso aplicar downgrade na versão do java)
- Dockerfile (front-end):
 - Node 20
- package.json:
 - vitejs 5.0.1
 - autoprefixer 10.4.21
 - jsdom 20.0.3
 - postcss 8.5.6
 - react 18.2.0
 - react-dom 18.2.0
 - tailwind css 3.4.17
 - vite 4.4.9

- pom.xml:
 - spring 3.5.5

4 - Implementação:

Back-end

Para realização dessa prova de conceito foram desenvolvidas 4 classes no back-end: controleContador, repositorioContador, servicoContador, modeloContador. Respectivamente, controleContador mapeia as rotas para os métodos listados em servicoContador para atender alguma requisição feita no front-end, repositorioContador permite realizar consultas ao banco de dados através do JPA, servicoContador contém a lógica para realização dos métodos necessários à aplicação e modeloContador é a entidade, na prática atua como uma relação que será manipulada na base de dados. A seguir serão apresentadas Figuras 1,2,3 e 4 com trechos das classes desenvolvidas.

```
@RestController
@RequestMapping("/api/contador")
@CrossOrigin(origins = "**") // permitir requisições do React
public class controleContador {

    private final servicoContador servico;

    public controleContador(servicoContador servico) {
        this.servico = servico;
    }

    @Autowired
    private repositorioContador repo;

    @GetMapping("/totalCliques") // funciona porque só tem um método que faz esse tipo de requisição -> se não
    // @GetMapping(qualquer coisa)
    public Long getTotalCliques() {
        return servico.getTotalCliques();
    }

    @PostMapping("/clicar")
    public Long incrementarCliques() {
        return servico.incrementarCliques();
    }

    @GetMapping("/status")
    public Map<String, Object> getStatus() {
        modeloContador clique = repo.findById(1L).orElseGet(() -> {
            modeloContador c = new modeloContador();
            c.setTotalCliques(0L);
            return repo.save(c);
        });
    }
}
```

Figura 1. controleContador

```
1 package com.provaConceito.provaConceito.repositorio;
2
3 import com.provaConceito.provaConceito.modelo.modeloContador;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface repositorioContador extends JpaRepository<modeloContador, Long> {
9 }
```

Figura 2. repositorioContador

```

29 package com.provaConceito.provaConceito.servico;
28
27 import com.provaConceito.provaConceito.repositorio.repositorioContador;
26 import com.provaConceito.provaConceito.modelo.modeloContador;
25
24 import org.springframework.stereotype.Service;
23
22 @Service
21 public class servicoContador {
20     private final repositorioContador repo;
19
18     public servicoContador(repositorioContador repo) {
17         this.repo = repo;
16     }
15
14     // Buscar o total de cliques (cria registro se não existir)
13     public Long getTotalCliques() {
12         return repo.findById(1L)
11             .map(modeloContador::getTotalCliques)
10             .orElseGet(() -> {
9                 modeloContador contador = new modeloContador();
8                 contador.setTotalCliques(0L);
7                 contador = repo.save(contador);
6                 return contador.getTotalCliques();
5             });
4     }
3

```

Figura 3. servicoContador

```

4 @Table(name = "modelo_contador")
3 @NoArgsConstructor
2 @AllArgsConstructor
1 public class modeloContador {
17     @Id
1     @GeneratedValue(strategy = GenerationType.IDENTITY)
2     private Long id;
3
4     public Long getId() {
5         return this.id;
6     }
7
8     public void setId(Long id) {
9         this.id = id;
10    }
11
12    @Column(nullable = false)
13    private Long totalCliques;
14
15    public Long getTotalCliques() {
16        return this.totalCliques;
17    }
18
19    public void setTotalCliques(Long totalCliques) {
20        this.totalCliques = totalCliques;
21    }
22 }

```

Figura 4. modeloContador

Front-end:

A principal implementação do front-end consiste na elaboração dos componentes BotaoCliques.jsx, Parabens.jsx e Modal.jsx, de resto não foge muito do padrão apresentado quando se inicia o projeto do zero utilizando o vite. Essencialmente, BotaoCliques requisita o back-end através de métodos HTTP a quantidade de cliques no banco de dados, atualização de dados no banco e realiza a atualização da página quando há pelo menos 10 cliques para o id 1

no banco. Parabens.jsx apenas exibe a nova tela com um texto escrito “Parabéns” em verde e Modal.jsx carrega um pop-up para realizar a transição entre as páginas.

Amarrando tudo junto:

As Figuras 5, 6, 7 e 8 ilustram o que é exibido na tela amarrando o front-end, back-end e o banco de dados, mostrando cada etapa da aplicação desenvolvida para a prova de conceito.

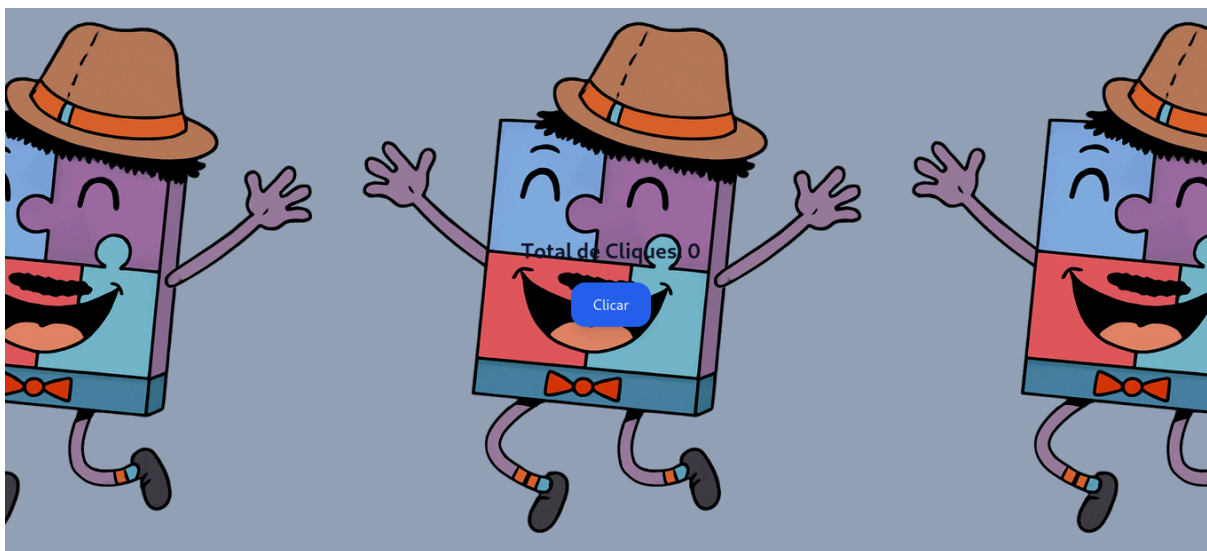


Figura 5. Tela inicial - sem dados no banco

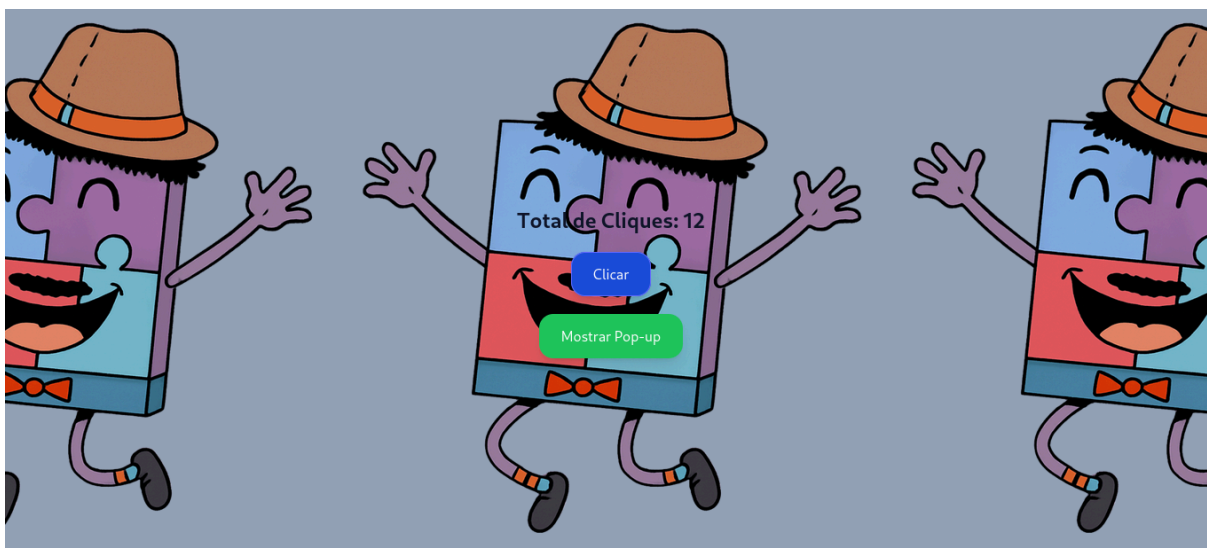


Figura 6. Tela apresentada quando há pelo menos 10 cliques no banco para o id 1



Figura 7. Pop-up apresentado antes de mudar de página

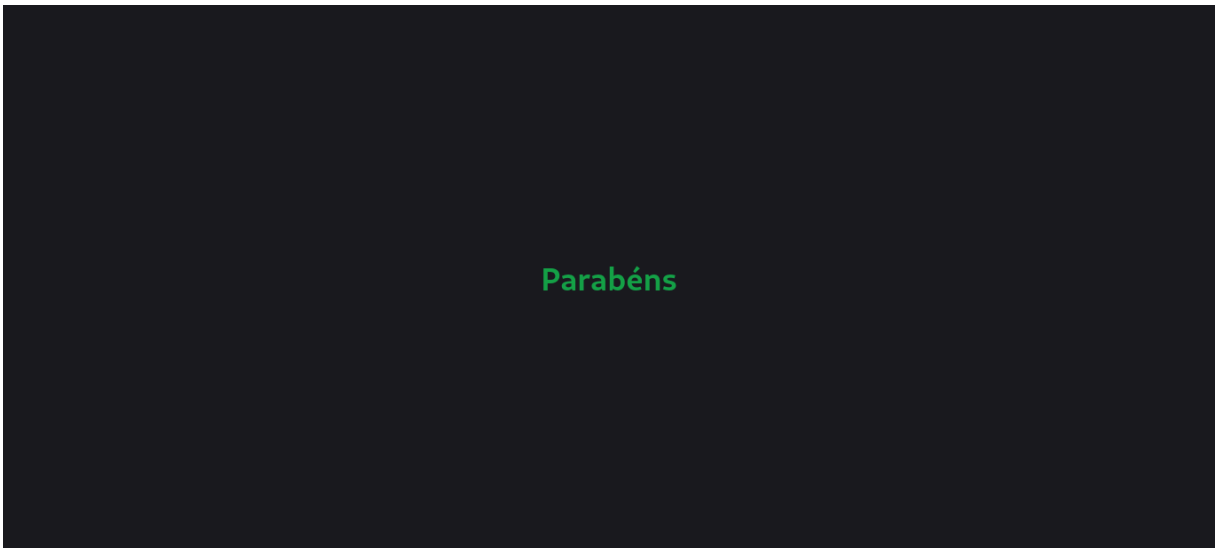


Figura 8. Tela atualizada

Implantação:

Para realizar a implantação da aplicação foi preciso mudar um pouco a estrutura de desenvolvimento da aplicação. Primeiramente, foi preciso editar o pom.xml do back-end para adicionar o perfil prod - permitindo que a aplicação gere tanto .war (deploy) quanto .jar (desenvolvimento). Sequencialmente, foi preciso criar a pasta webapp no mesmo nível das pastas java e resources e inserir nela o conteúdo produzido pelo comando `docker compose run --rm frontend npm build`. Além disso, também foi preciso alterar os caminhos das rotas nos arquivos BotaoCliquess.jsx, controleContador.java, [vite.config.js](#) e main.jsx - estes caminhos que devem ser mudados manualmente dependendo do contexto (desenvolvimento ou implantação). Após cumprir essas etapas, basta compilar o back-end usando o comando `docker compose exec -it backend mvn clean package -Pprod` enquanto o container do

back-end estiver em execução. Por segurança, os detalhes da implantação no servidor serão omitidos nesse documento.

Conclusão:

Essa documentação apresenta mais uma perspectiva do que foi descrito no documento Ambiente de desenvolvimento. Este documento buscou complementar algumas informações que ao meu ver se perderam no documento supracitado nesta seção, contemplando a versão das tecnologias e o que foi produzido de fato nesta prova. Mesmo com estes dois documentos ainda não é possível apresentar e destrinchar todos os detalhes da aplicação sem escrever um documento gigantesco, por esta razão é válido visitar o repositório e estudar os arquivos listados neste documento para compreender melhor o que faz cada função, notação, classe e componente usados nesta prova.